

Thuật toán mới cho Half-plane Intersection và giá trị thực tiễn

Zeyuan Zhu

29 tháng 12 năm 2005

Nguồn gốc: New algorithm for Half-plane Intersection and its Practical Value

IOI Candidate Team Papers/2006/Zeyuan Zhu/Zeyuan Zhu.pdf

Luận văn cho kỳ tuyển chọn đội tuyển Trung Quốc năm 2006

Zeyuan Zhu, lớp 12, Nanjing Foreign Language School, Jiangsu, China

Tặng người mẹ yêu quý của tôi, Xiaoli Xu.

E-mail: zhuzeyuan[at]hotmail[dot]com

Từ khóa: Half-plane, Intersection, Feasible Region, Algorithm, Polygon, Practical.

Abstract

Bài viết trình bày một thuật toán $O(n \log n)$ mới cho **Half-plane Intersection** (viết tắt là HPI), một trong các bài toán được thảo luận nhiều trong khoa học máy tính. Trọng tâm là giá trị của thuật toán trong ứng dụng thực tế; ở một mức độ nhất định có thể hạ độ phức tạp xuống $O(n)$, trong khi thuật toán vẫn rất dễ cài đặt.

Mục 1 giới thiệu bài toán HPI. Mục 2 chuẩn bị một thuật toán tuyến tính cho **Convex Polygon Intersection** (CPI). Dựa trên đó, mục 3 nhắc lại ngắn gọn lời giải HPI phổ biến bằng **Divide-and-Conquer** (D&C). Mục 4 trình bày chi tiết thuật toán mới **Sort-and-Incremental** (S&I). Mục 5 vừa tổng kết vừa bàn về ứng dụng thực tế và so sánh với thuật toán cũ ở mục 3.

Mốc thời gian. Ý tưởng xuất hiện vào tháng 4 năm 2005; một phần được cài đặt vào tháng 6 năm 2005¹; một bài toán được ra vào tháng 7 năm 2005²; công bố dưới dạng bài viết trên USENET ngày 6 tháng 11 năm 2005³

1 Giới thiệu

Một đường thẳng trong mặt phẳng thường được biểu diễn bởi

$$ax + by = c.$$

Tương tự, dạng bất đẳng thức

$$ax + by \leq c$$

biểu diễn một nửa mặt phẳng, tức một phía của đường thẳng đó. Cần chú ý rằng $ax + by \leq c$ và $-ax - by \leq -c$ là hai nửa mặt phẳng đối nhau, khác với trường hợp đường thẳng $ax + by = c$ và $-ax - by = -c$ biểu diễn cùng một đường.

¹Bài toán con của HPI xuất hiện trong kỳ thi USA Invitational Computing Olympiad.

²Tác giả đã ra một bài HPI trên Peking University Online Judge, kèm giới thiệu ngắn về thuật toán.

³<http://groups.google.com/group/sci.math/msg/68e9d9fc634f0d92>.

Bài toán **Half-plane Intersection** (HPI) được phát biểu như sau: cho n nửa mặt phẳng

$$a_i x + b_i y \leq c_i, \quad 1 \leq i \leq n,$$

hãy xác định tập tất cả các điểm thỏa mãn đồng thời n bất đẳng thức đó.

Miền khả thi, tức giao của các nửa mặt phẳng, có dạng một miền lồi và có thể không bị chặn. Trong thực hành ta thường thêm bốn nửa mặt phẳng tạo thành một hình chữ nhật đủ lớn để bảo đảm diện tích sau giao là hữu hạn. Trong các mục sau, ta giả sử miền khả thi bị chặn và có diện tích hữu hạn.

Mỗi nửa mặt phẳng tạo ra nhiều nhất một cạnh của đa giác lồi kết quả, nên miền lồi thu được có nhiều nhất n cạnh. Tuy vậy, giao cũng có thể suy biến thành một đường thẳng, một tia, một đoạn thẳng, một điểm, hoặc là rỗng.

2 Convex Polygon Intersection (CPI)

Bài toán giao hai đa giác lồi A và B thành một đa giác lồi mới có thể giải bằng Line Segment Intersection trong thời gian $O(n \log n)$, khi tổng số cạnh là $O(n)$. Ở đây ta phác thảo một cách đơn giản hơn và hiệu quả hơn: phương pháp quét mặt phẳng.

Ý tưởng chính là tính các giao điểm của cạnh làm điểm cắt, rồi chia biên của A và B thành các đoạn nằm ngoài và các đoạn nằm trong đa giác kia. Các đoạn biên nằm trong được nối với nhau, tạo thành đường bao của đa giác giao. Trong hình minh họa của bản gốc, các đoạn biên trong được tô đậm và chính là đường bao cần tìm.

Giả sử có một đường quét thẳng đứng đi từ trái sang phải. Các hoành độ được đường quét xử lý gọi là **sự kiện** x . Tại mọi thời điểm, đường quét có nhiều nhất bốn giao điểm với hai đa giác đã cho⁴:

- giao với vỏ trên của A , ký hiệu A_u ;
- giao với vỏ dưới của A , ký hiệu A_l ;
- giao với vỏ trên của B , ký hiệu B_u ;
- giao với vỏ dưới của B , ký hiệu B_l .

Trên đường quét hiện tại, điểm thấp hơn trong hai điểm A_u, B_u và điểm cao hơn trong hai điểm A_l, B_l tạo thành đoạn của miền giao hiện tại. Nếu đoạn này không rỗng thì nó nằm trong cả hai đa giác.

Dĩ nhiên ta không thể quét qua mọi hoành độ hữu tỉ. Gọi các cạnh chứa A_u, A_l, B_u, B_l lần lượt là e_1, e_2, e_3, e_4 . Sự kiện x kế tiếp được chọn trong các đầu mút của bốn cạnh này và bốn giao điểm tiềm năng

$$e_1 \cap e_3, \quad e_1 \cap e_4, \quad e_2 \cap e_3, \quad e_2 \cap e_4.$$

Thao tác trên có thể cài đặt trong thời gian $O(n)$, vì chỉ có $O(n)$ sự kiện x , còn việc duy trì A_u, A_l, B_u, B_l tốn $O(1)$ cho mỗi sự kiện.

3 Lời giải phổ biến: Divide-and-Conquer (D&C)

Cách tiếp cận cơ bản dựa trên chia để trị.

Chia: Chia n nửa mặt phẳng thành hai tập có kích thước $\lfloor n/2 \rfloor$ và $\lceil n/2 \rceil$.

Trị: Tính đệ quy miền khả thi, tức giao, của từng tập con.

⁴Giả sử không có cạnh nào song song với đường quét. Nếu có, ta có thể xoay mặt phẳng một góc thích hợp; nếu không xoay, cần xử lý nhiều trường hợp biên.

Gộp: Tính giao của hai đa giác lồi thu được bằng thuật toán CPI tuyến tính ở mục 2.

Độ phức tạp thời gian tổng cộng được suy ra từ phương trình truy hồi

$$T(n) = 2T(n/2) + O(n),$$

nên

$$T(n) = O(n \log n).$$

4 Thuật toán mới: Sort-and-Incremental (S&I)

Trước hết ta định nghĩa góc cực của một nửa mặt phẳng. Chẳng hạn:

- với nửa mặt phẳng dạng $x - y \geq$ hằng số, góc cực được định nghĩa là $\pi/4$;
- với nửa mặt phẳng dạng $x + y \leq$ hằng số, góc cực được định nghĩa là $3\pi/4$;
- với nửa mặt phẳng dạng $x + y \geq$ hằng số, góc cực được định nghĩa là $-\pi/4$;
- với nửa mặt phẳng dạng $x - y \leq$ hằng số, góc cực được định nghĩa là $-3\pi/4$.

Nói cách khác, ta gán cho mỗi nửa mặt phẳng một hướng của đường biên để các nửa mặt phẳng có thể được sắp theo góc cực. Với nửa mặt phẳng dạng $ax + by \leq c$, ta gọi c là hằng số của nó.

Thuật toán Sort-and-Incremental được mô tả chi tiết như sau.

Bước 1. Chia các nửa mặt phẳng thành hai tập. Một tập có góc cực trong $(-\pi/2, \pi/2]$; tập còn lại có góc cực trong

$$(-\pi, -\pi/2] \cup (\pi/2, \pi].$$

Thực ra chỉ cần chia thành hai khoảng độ dài π , chẳng hạn $(a, a + \pi]$ và $(a + \pi, a + 2\pi]$. Cách chia trên được dùng để giải thích rõ hơn.

Bước 2. Xét tập có góc cực trong $(-\pi/2, \pi/2]$; tập còn lại được xử lý tương tự ở bước 3 và bước 4. Sắp xếp các nửa mặt phẳng theo góc cực. Với các nửa mặt phẳng có cùng góc cực, chỉ giữ lại nửa mặt phẳng chặt nhất, tức nửa mặt phẳng bị các nửa mặt phẳng còn lại phủ lên; các nửa mặt phẳng khác có thể xóa đi. Nếu tất cả được viết cùng dạng $ax + by \leq c$ với cùng hướng, đó là nửa mặt phẳng có hằng số nhỏ nhất.

Bước 3. Bắt đầu với hai nửa mặt phẳng có góc cực nhỏ nhất sau khi sắp xếp. Tính giao điểm hai đường biên của chúng và đẩy cả hai nửa mặt phẳng vào một ngăn xếp.

Sau đó, mỗi lần thêm một nửa mặt phẳng mới theo thứ tự góc cực tăng dần. Tính giao điểm giữa nửa mặt phẳng mới và nửa mặt phẳng ở đỉnh ngăn xếp. Nếu giao điểm này nằm bên phải giao điểm của hai nửa mặt phẳng trên cùng trong ngăn xếp, ta loại đỉnh ngăn xếp. Tiếp tục kiểm tra với hai nửa mặt phẳng mới ở đỉnh ngăn xếp; nếu giao điểm hiện tại vẫn nằm bên phải giao điểm ở đỉnh, lại loại tiếp. Lặp cho đến khi giao điểm hiện tại nằm bên trái giao điểm ở đỉnh, rồi đẩy nửa mặt phẳng hiện tại vào ngăn xếp.

Hình minh họa trong bản gốc cho thấy trực giác của thao tác này: khi thêm một nửa mặt phẳng có góc cực lớn hơn, các giao điểm cũ nằm bên trái giao điểm mới không còn tạo thành phần biên hợp lệ của nửa đa giác đang xây dựng, nên các nửa mặt phẳng tương ứng bị pop khỏi ngăn xếp.

Bước 4. Các giao điểm của những cặp nửa mặt phẳng kề nhau trong ngăn xếp tạo thành một nửa của đa giác lồi. Với hai tập ở bước 1, ta thu được hai nửa của đa giác: $(-\pi/2, \pi/2]$ cho vỏ trên, còn $(-\pi, -\pi/2] \cup (\pi/2, \pi]$ cho vỏ dưới. Có thể gộp chúng trong thời gian tuyến tính bằng CPI ở mục 2, nhưng ta còn có thể tận dụng cách lập sau.

Dựa trên cấu trúc hàng đợi hai đầu, ta liên tục loại một đầu mút đã biết là không khả thi. Ban đầu bốn con trỏ p_1, p_2, p_3, p_4 trở tới các cạnh trái nhất và phải nhất của vỏ trên và vỏ dưới. Hai con trỏ p_1, p_3 đi sang phải, còn p_2, p_4 đi sang trái. Tại mọi thời điểm, nếu giao điểm trái nhất không thỏa nửa mặt phẳng do p_1 hoặc p_3 cung cấp, giao điểm trái nhất đó chắc chắn phải bị xóa; con trỏ tương ứng đi sang cạnh kề bên phải. Tương tự, ta xử lý giao điểm phải nhất với p_2 và p_4 .

Mỗi lần chỉ thử xóa một giao điểm trái nhất hoặc phải nhất. Lý do là có thể giao điểm ngoài cùng vi phạm một nửa mặt phẳng, trong khi giao điểm kế bên lại nằm đúng phía của một nửa mặt phẳng khác và không thể xóa cùng lúc. Lập quá trình này cho đến khi không còn cập nhật nào. Trong quá trình đó, số giao điểm của một vỏ riêng lẻ có thể giảm xuống 0, nhưng miền khả thi tổng thể vẫn có thể không rỗng.

Ngoại trừ bước sắp xếp ở bước 2, mọi phần của thuật toán S&I đều tuyến tính, tức $O(n)$. Nếu dùng quick-sort cho bước 2, độ phức tạp tổng cộng là $O(n \log n)$, với hằng số ẩn khá nhỏ.

5 Giá trị thực tiễn và cách tiếp cận tuyến tính

Những ý tưởng lớn cần cả cánh để bay lẫn bộ phận hạ cánh để đáp xuống thực tế. Thuật toán S&I có vẻ cùng độ phức tạp thời gian với D&C, nhưng khi cài đặt nó có một số ưu thế rõ rệt.

- Chương trình S&I dễ viết hơn nhiều so với chương trình D&C. Bản cài đặt bằng C++ dài chưa tới 3 KB.
- Hằng số ẩn trong độ phức tạp của S&I nhỏ hơn đáng kể so với D&C, vì ta không còn cần $O(n \log n)$ phép tính giao điểm. Nói chung, chương trình S&I chạy nhanh hơn chương trình D&C xấp xỉ năm lần.
- Phép tính giao điểm là thao tác rất quan trọng trong cả hai thuật toán vì nó chậm, tương đương hàng trăm phép cộng. CPI, thuật toán chuẩn bị được D&C gọi nhiều lần, cần $O(n)$ phép tính giao điểm ở mỗi lần gộp, khiến tổng thời gian tăng lên. Ngược lại, S&I chỉ tính giao điểm $O(n)$ lần trong mọi trường hợp.
- Nếu các nửa mặt phẳng đầu vào đều nằm trong $(-\pi/2, \pi/2]$, hoặc trong bất kỳ khoảng góc nào có độ dài π , chương trình S&I có thể được rút ngắn đáng kể, chỉ khoảng hai mươi dòng C++. D&C không có lợi thế tương tự. Một bài toán trong kỳ USA Invitational Computing Olympiad đã khai thác đúng tình huống này.

Tối ưu tiệm cận xuống tuyến tính. Nút thắt của thuật toán là sắp xếp. Cần chú ý rằng phép sắp xếp ở đây không nhất thiết là sắp xếp so sánh. Khóa của các phần tử cần sắp là góc cực, có thể xác định bởi hệ số góc, tức một phân số thập phân. Do đó ta có thể thay quick-sort $O(n \log n)$ bằng radix-sort $O(n)$, và hạ độ phức tạp tiệm cận của thuật toán xuống $O(n)$. Dẫu vậy, trong thực tế cách $O(n)$ thường có thể chạy chậm hơn cách $O(n \log n)$ vì cần thêm bộ nhớ.

Cảm nghĩ về sáng tạo. Một phát minh không thuộc về người chỉ nảy ra ý tưởng. Phần lớn mọi người đều có ý tưởng. Khác biệt giữa một người bình thường và một nhà phát minh là vì lý do nào đó nhà phát minh tin vào chính mình và có thôi thúc đưa ý tưởng đến kết quả. Henri Matisse, họa sĩ và nhà điêu khắc người Pháp, từng nói rằng không có gì khó hơn đối với một họa sĩ thật sự sáng tạo hơn là vẽ một bông

hồng, bởi trước khi làm được điều đó, người ấy phải quên tất cả những bông hồng từng được vẽ. Mang theo cả tinh thần đổi mới lý tưởng lẫn thực tiễn, tôi đang trên đường. Còn bạn thì sao?

References

- [1] Kurt Mehlhorn. *Data Structures and Algorithms, Vol. 3*. Springer-Verlag, New York, 1984.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 2nd edition. MIT Press, New York, 2001.
- [3] Gill Barequet. Computational Geometry, Lecture 4, Spring 2004/2005.
- [4] Kavitha Telikepalli. Algorithms and Data Structures, Lecture 3, Winter 2003/2004.